Laboratorio 1 · Modelado OLTP → OLAP

SI7006 · Almacenamiento y Procesamiento de Grandes Datos · EAFIT 2026-2

Estudiante: David Lopera Londoño

El objetivo de este laboratorio es pasar el esquema transaccional de PayTrack (OLTP, en tercera forma normal) a un esquema estrella (OLAP), cargarlo con un proceso ELT y comparar, con planes de ejecución, cómo responde cada modelo tres preguntas de negocio: el monto por categoría de comercio en los últimos 30 días (Q1), la tasa de fraude por segmento de cliente (Q2) y el volumen de transacciones por día de la semana y hora en los últimos 7 días (Q3).

1. Diagnóstico del OLTP (Parte A)

La Tabla 1 resume lo que muestran los planes de ejecución del OLTP con 50.000 transacciones: qué pasos consumen más tiempo y por qué el diseño normalizado obliga a hacerlos.

Consulta	Qué domina el plan	Por qué la 3FN obliga a ese trabajo
Q1 20,0 ms	Usa el índice de fecha para encontrar las transacciones del mes, pero como están repartidas por toda la tabla termina leyendo sus **569 páginas**. Después las une con merchants y agrupa.	La categoría no está en transactions sino en merchants, así que hay que unir ambas tablas. Además, el estado es texto sin índice y se revisa fila por fila.
Q2 33,4 ms	Lee las 50.000 transacciones y las une con customers. Ese join, hecho sobre códigos de texto, se lleva la mitad del tiempo (~17 ms).	El segmento está en customers, y además solo se guarda el **actual**: si un cliente cambió de segmento, sus transacciones antiguas quedan mal clasificadas.
Q3 10,2 ms	Vuelve a leer toda la tabla, calcula el día y la hora de cada fila con EXTRACT() y ordena las 4.291 filas para poder agruparlas.	Solo se guarda la fecha y hora completa, así que el día y la hora se recalculan en cada consulta. El optimizador esperaba **4.297 grupos y fueron 168**, por eso eligió ordenar.

Tabla 1. Planes de ejecución del OLTP con 50.000 transacciones.

¿Qué pasaría con 500K transacciones diarias durante un año? La tabla tendría 182,5 millones de filas, 3.650 veces más que ahora. Si los tiempos crecieran en la misma proporción, Q2 tardaría unos 2 minutos, porque siempre recorre toda la historia, y Q1 unos 15 segundos. En Q3, ordenar las filas de la semana necesitaría unos 320 MB de memoria, muy por encima de los 4 MB que PostgreSQL asigna por defecto, así que tendría que ordenar en disco; de hecho, esto ya ocurre con 1 millón de filas (Parte D). Y todo esto correría en la misma base que procesa los pagos, quitándole recursos a la operación.

2. Diseño del esquema estrella (Parte B)

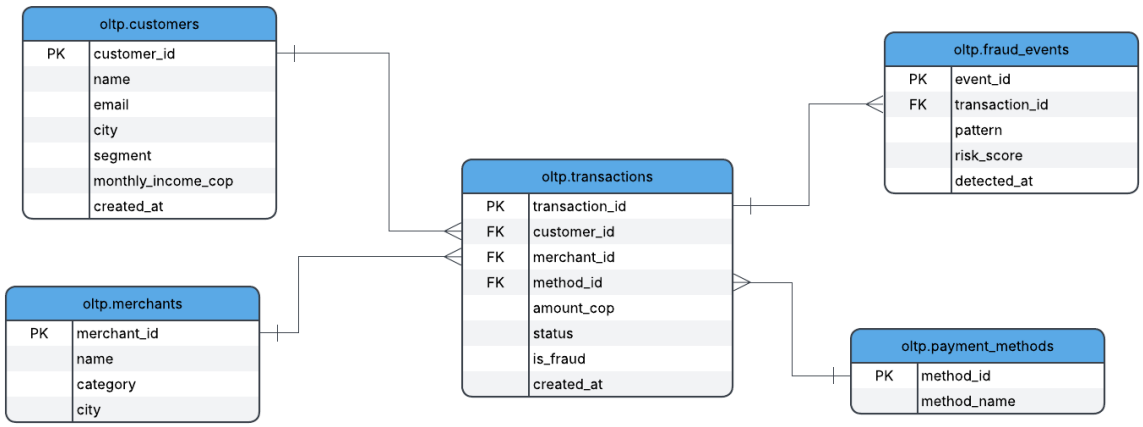


Figura 1a. Antes: esquema transaccional oltp (3FN).

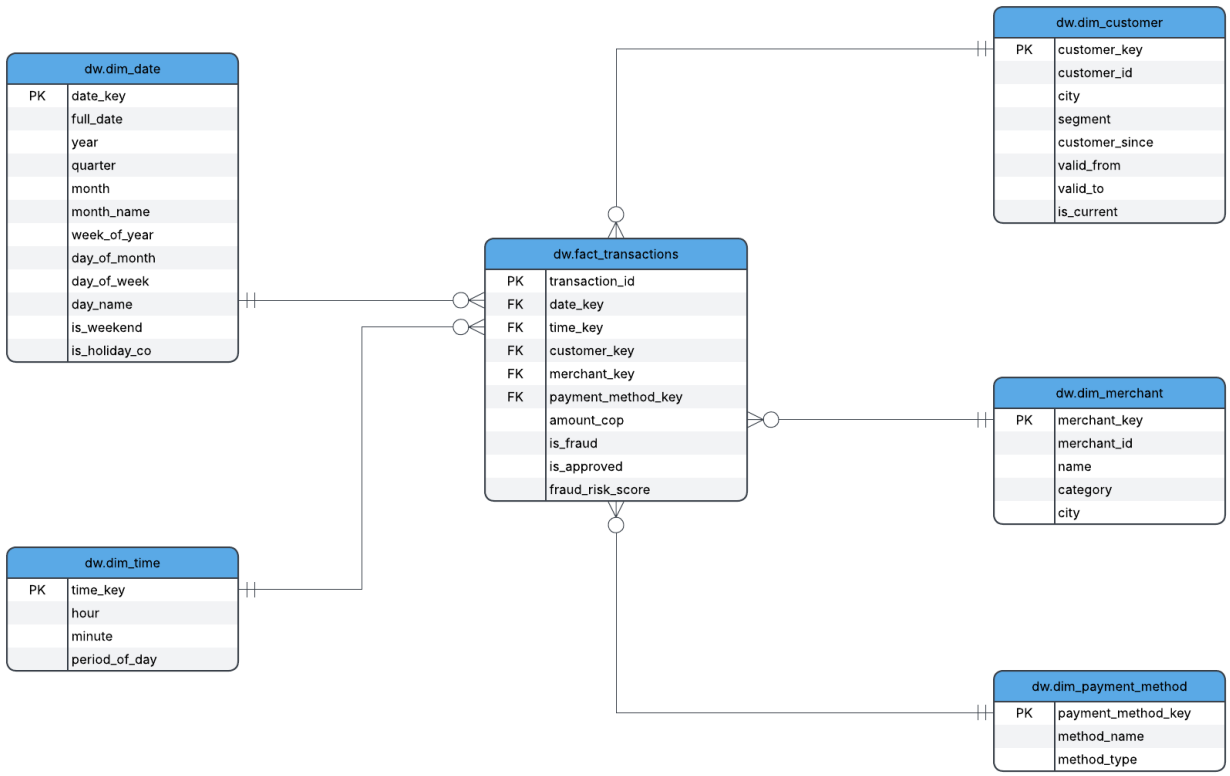


Figura 1b. Después: esquema estrella dw. Grano de fact_transactions: una fila por transacción.

Proceso de negocio y grano. El proceso que se modela es el procesamiento de pagos: un cliente le paga a un comercio usando un método de pago. Cada fila de la tabla de hechos representa **una transacción**, ya sea aprobada o rechazada. Se eligió este grano porque es el nivel más detallado posible y desde ahí se puede responder cualquier pregunta sumando hacia arriba (por día, por comercio, por segmento). El transaction_id se conserva en el hecho como llave primaria, lo que impide que una misma transacción quede cargada dos veces.

Dimensiones. Cada dimensión responde una pregunta sobre la transacción: dim_date y dim_time dicen cuándo ocurrió; dim_customer dice quién pagó (ciudad, segmento y fecha de alta); dim_merchant dice a quién se le pagó (nombre, categoría y ciudad); y dim_payment_method dice cómo se pagó, e incluye method_type (tarjeta, billetera digital o transferencia), un atributo que no existía en el OLTP. Algunos datos se dejaron fuera a propósito: el nombre y el email del cliente son datos personales que no sirven para analizar y la Ley 1581 de 2012 pide no replicarlos sin necesidad; el ingreso mensual es un dato financiero sensible, y el segmento ya refleja el perfil del cliente. Del detalle de fraud_events solo se trae el puntaje de riesgo, porque el resto describe otro proceso (la detección de fraude).

Medidas. El monto (amount_cop) es aditivo: se puede sumar por cualquier dimensión. is_fraud e is_approved también lo son si se cuentan, por ejemplo para saber cuántos fraudes hubo por segmento. El puntaje de riesgo (fraud_risk_score) es no aditivo, porque sumar probabilidades no tiene sentido: se promedia o se toma el máximo, y queda vacío cuando la transacción no tuvo evento de fraude. No hay medidas semiaditivas, que aparecen con valores como un saldo, y aquí cada fila es un pago.

Surrogate keys. Cada dimensión usa una llave numérica propia del data warehouse en lugar del código del OLTP. La razón principal es el SCD2: un mismo cliente puede tener varias filas, una por cada versión, así que customer_id se repite y no puede identificar la fila. Además, estas llaves desacoplan el DW de los códigos del sistema fuente y, al ser enteros, hacen los joins más baratos que con los códigos de texto del OLTP.

SCD Tipo 2 en cliente. El segmento de un cliente puede cambiar con el tiempo, por ejemplo de mass a premium, y también su ciudad. Si el valor simplemente se sobrescribiera, todas sus transacciones pasadas quedarían como premium, y un fraude cometido cuando era mass se contaría como fraude premium, lo que distorsiona la tasa de fraude por segmento (Q2). Con SCD2 cada cambio crea una nueva versión del cliente con su periodo de vigencia, y cada transacción queda ligada a la versión que estaba vigente cuando ocurrió. Por eso Q2 no filtra por is_current: cada hecho ya apunta al segmento correcto.

Implementación y carga (Parte C). Al traer el puntaje de fraude se toma el máximo por transacción, para que una transacción con dos eventos no aparezca duplicada. Además, la tabla de hechos se carga ordenada por fecha, como ocurriría con una carga diaria; esa decisión resultó importante en la Parte D. La carga pasa las 9 validaciones de load_star.py y check_queries.py confirma que las tres consultas dan los mismos resultados en ambos modelos (las capturas se entregan aparte).

3. Comparación de rendimiento (Parte D)

Las tres consultas se midieron en ambos modelos con 50K, 500K y 1M transacciones, y con 2M como extensión. En todos los casos se tomó la segunda ejecución, después de actualizar las estadísticas con ANALYZE.

Consulta	Modelo	50K	500K	1M	2M
Q1	OLTP	20,0	137,3	231,2	424,5
	OLAP	35,3	138,4	260,4	430,0
Q2	OLTP	33,4	138,5	339,4	595,9
	OLAP	29,3	140,1	312,7	579,6
Q3	OLTP	10,2	100,7	353,0	733,3
	OLAP	**5,0**	**49,9**	**125,6**	**163,1**

Tabla 2. Execution Time (ms) de cada consulta según el volumen de datos.

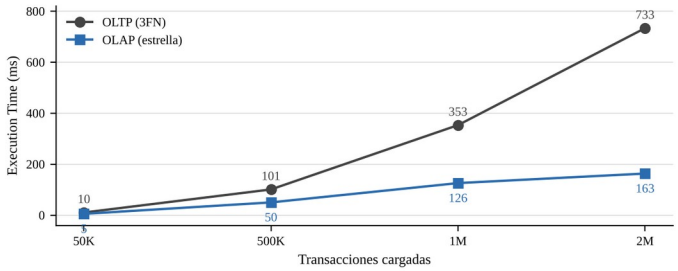


Figura 2. Crecimiento del tiempo de Q3 en cada modelo.

Q3 es donde el modelo estrella marca la diferencia: es de 2 a 4,5 veces más rápido, y la brecha crece con el volumen. Como la tabla de hechos está ordenada por fecha, el optimizador usa el índice y lee solo las filas de la última semana en lugar de toda la tabla. El día y la hora ya vienen calculados en las dimensiones, y al final solo tiene que ordenar los 168 resultados. El OLTP, en cambio, ordena todas las filas de la semana, y desde 1 millón de filas ya no le caben en memoria y lo hace en disco.

En Q1 y Q2 los dos modelos quedan prácticamente empatados. Q1 necesita un tercio de las filas y Q2 todas, así que en ambos casos los dos modelos terminan leyendo la tabla completa. En Q1 el OLTP tiene una pequeña ventaja, porque filtrar por fecha directamente es más barato que hacer el join con dim_date. En Q2 el OLAP gana entre 3 y 8 %, gracias a que une por números en lugar de texto y a que su tabla de hechos es más liviana.

Experimento con índices

Índice	¿Se usa?	Qué se observó
idx_fact_date	Solo en Q3, y solo con la tabla ordenada	Mientras las filas estaban desordenadas, el optimizador nunca lo usó. Al cargar la tabla ordenada por fecha, Q3 con 1M pasó de 207,8 a 125,6 ms. Con 50K, Q3 tarda 5,3 ms con el índice y 14,8 ms sin él.
idx_fact_merchant, idx_fact_customer	No	Las consultas que unen con comercios o clientes necesitan casi toda la tabla, y para eso es más barato leerla completa.

Tabla 3. Uso de los índices de fact_transactions.

No todos los índices ayudan. Un índice sirve para encontrar pocas filas; cuando la consulta necesita una parte grande de la tabla, o las filas que busca están repartidas por todas partes, al optimizador le sale más barato leerla de corrido (sequential scan), y además en paralelo, que saltar de página en página siguiendo el índice.

Conclusión

El modelo estrella genera planes más simples y eficientes en todas las consultas, y escala claramente mejor cuando la pregunta es selectiva, como Q3. Cuando hay que recorrer casi toda la tabla, la ventaja se diluye, porque PostgreSQL guarda y procesa los datos fila por fila y los dos modelos terminan haciendo un trabajo parecido.